

סרטייה

מערכת השאלת סרטי וידאו/DVD

מסמך תכנון

פרט	מידע
מגיש	אחמד עבד אל חלים
קורס	סדנה בתכנות מתקדם בשפת Java - 20503
מוסד	האוניברסיטה הפתוחה
תאריך	אוגוסט 2026

1. תיאור כללי
2. מינוחים
3. הטכנולוגיות שבשימוש
4. ארכיטקטורת המערכת
5. בסיס הנתונים
6. מבנה החבילות
7. שכבת המודל
8. שכבת הנתונים
9. שכבת הלוגיקה העסקית
10. שכבת התצוגה
11. שכבת ה-REST
12. טיפול במקביליות
13. אבטחה
14. החלטות תכנון והנימוק להן

1. תיאור כללי

מסמך זה מתאר את המבנה הפנימי של מערכת "סרטייה", את הטכנולוגיות שבהן מומשה ואת המחלקות והקשרים ביניהן.

המערכת בנויה בארכיטקטורה שכבתית בהתאם לעקרונות MVC, על גבי Jakarta (Java) EE 10, ומשתמשת בבסיס נתונים MySQL. הדגש התכנוני המרכזי הוא הפרדה בין השכבות ונכונות הפעולה בתנאי ריבוי משתמשים.

2. מינוחים

מונח	משמעות
(Movie) סרט	כותר בקטלוג. אינו ניתן להשאלה בעצמו.
עותק (Copy)	דיסק פיזי אחד של סרט. זוהי היחידה המושאלת בפועל.
(Rental) השאלה	רשומה המתעדת שעותק יצא ללקוח. "פתוחה" כל עוד לא הוחזר.
לקוח	משתמש רשום השואל סרטים.
מנהל	משתמש בעל הרשאות ניהול קטלוג ומלאי.

ההבחנה בין סרט לעותק היא ההחלטה המבנית החשובה ביותר במערכת. היא מאפשרת להחזיק כמה עותקים של כותר מבוקש, ובו בזמן לאכוף במדויק שעותק מסוים לא יושאל פעמיים.

3. הטכנולוגיות שבשימוש

הפרויקט מממש שלוש טכנולוגיות מרכזיות:

3.1 JSF — Jakarta(Java) Faces 4.0

JSF היא מסגרת העבודה של ממשק המשתמש. דפי XHTML משתמשים ב-Expression Language ומקושרים ל-Backing Beans בצד השרת. נעשה שימוש בתבנית משותפת (ui:composition / ui:insert), ברכיבי טפסים, ב-`ui:repeat`, בהמרות תאריך ומספר, בהצגת הודעות ובאימות קלט. ניהול המצב נעשה באמצעות `ViewScoped@` ו-`SessionScoped@`.

3.2 REST — Jakarta(Java) RESTful Web Services 3.1 (JAX-RS)

ממשק תכנות המחזיר JSON בכתובת `/api`. הוא חושף את הקטלוג לקריאה, משתמש באותה שכבת שירות של דפי ה-JSF עבור פעולות הקריאה, וכולל רכיבי `ExceptionHandler` הממירים חריגות עסקיות לקודי HTTP מתאימים.

3.3 CDI 4.0

CDI משמש להזרקת תלויות וניהול מחזור החיים של Beans ושל שירותים. כך רכיבי התצוגה והשירותים אינם יוצרים את התלויות שלהם ידנית, והקשרים ביניהם נשארים מפורשים וניתנים לבדיקה.

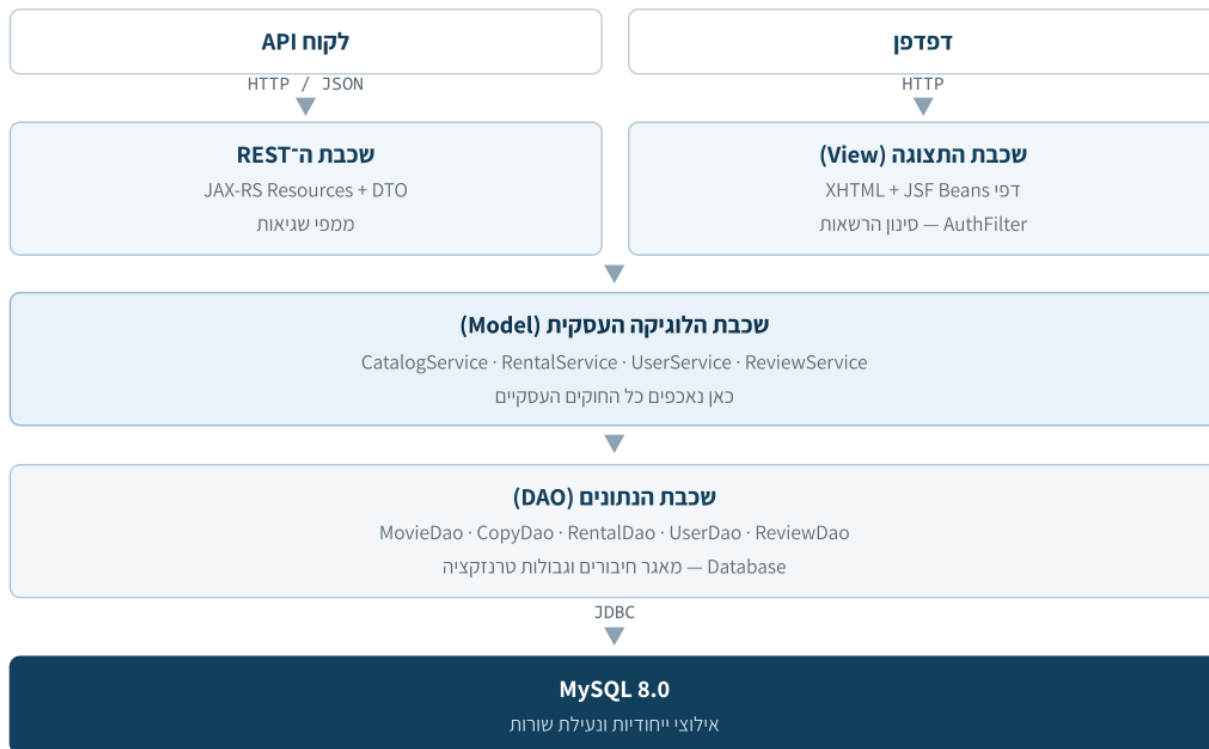
3.4 טכנולוגיות תומכות

רכיב	גרסה	תפקיד
Java	17 (LTS)	שפת המימוש
Jakarta EE	10	Faces, CDI, JAX-RS, Servlet
JDBC	4.x	גישה ישירה לבסיס הנתונים וטרנזקציות
MySQL	8.0	בסיס הנתונים
HikariCP	5.1	מאגר חיבורים
Payara Micro	6.2024.6	שרת היישומים
Maven	3.9	בנייה וניהול תלויות
JUnit 5	5.10	בדיקות

הבחירה ב-JDBC ולא ב-JPA מכוונת: מנגנון המקביליות המרכזי משתמש בנעילת שורות מפורשת באמצעות `SELECT ... FOR UPDATE SKIP LOCKED`, ולכן ניסוח הנעילה ישירות ב-SQL ברור ומדויק, וגבולות הטרנזקציה נשארים מפורשים.

4.1 תרשים כללי

4.1 תרשים כללי



תרשים כללי — שתי שכבות כניסה מעל שכבת שירות משותפת ושכבת DAO/JDBC

4.2 כללי ההפרדה בין השכבות

- שכבת התצוגה אינה ניגשת ל-DAO. ה-Backing Beans אינם מייבאים מחבילת dao; כל פעולה עוברת דרך שירות.
- שכבת השירות אינה תלויה ב-HTTP. אין בה `HttpServletRequest`, `FacesContext` או קודי סטטוס. היא זורקת חריגות עסקיות, ושכבת הכניסה מתרגמת אותן: JSF להודעה על המסך ו-JAX-RS לקוד HTTP.
- שכבת ה-DAO אינה מכירה חוקים עסקיים. היא מבצעת שאילתות ומחזירה אובייקטי מודל. ההחלטה האם מותר להשאיל נמצאת בשירות בלבד.
- דפי ה-JSF ומשאבי ה-REST משתמשים באותה שכבת שירות בפעולות המשותפות להם. ה-REST הציבורי מוגבל לקריאה בלבד, ולכן פעולות שינוי מלאי נשארות במסלול המאומת של ממשק ה-JSF.

5.1 גישה

בסיס הנתונים הוא MySQL 8.0 המאזין על פורט 3306. הגישה אליו נעשית בתקן JDBC דרך מחלקת Database המחזיקה מאגר חיבורים. פרטי החיבור נמצאים בקובץ `sartia.properties`, וניתן לעקוף את ערכי ברירת המחדל באמצעות פרמטר הרצה.

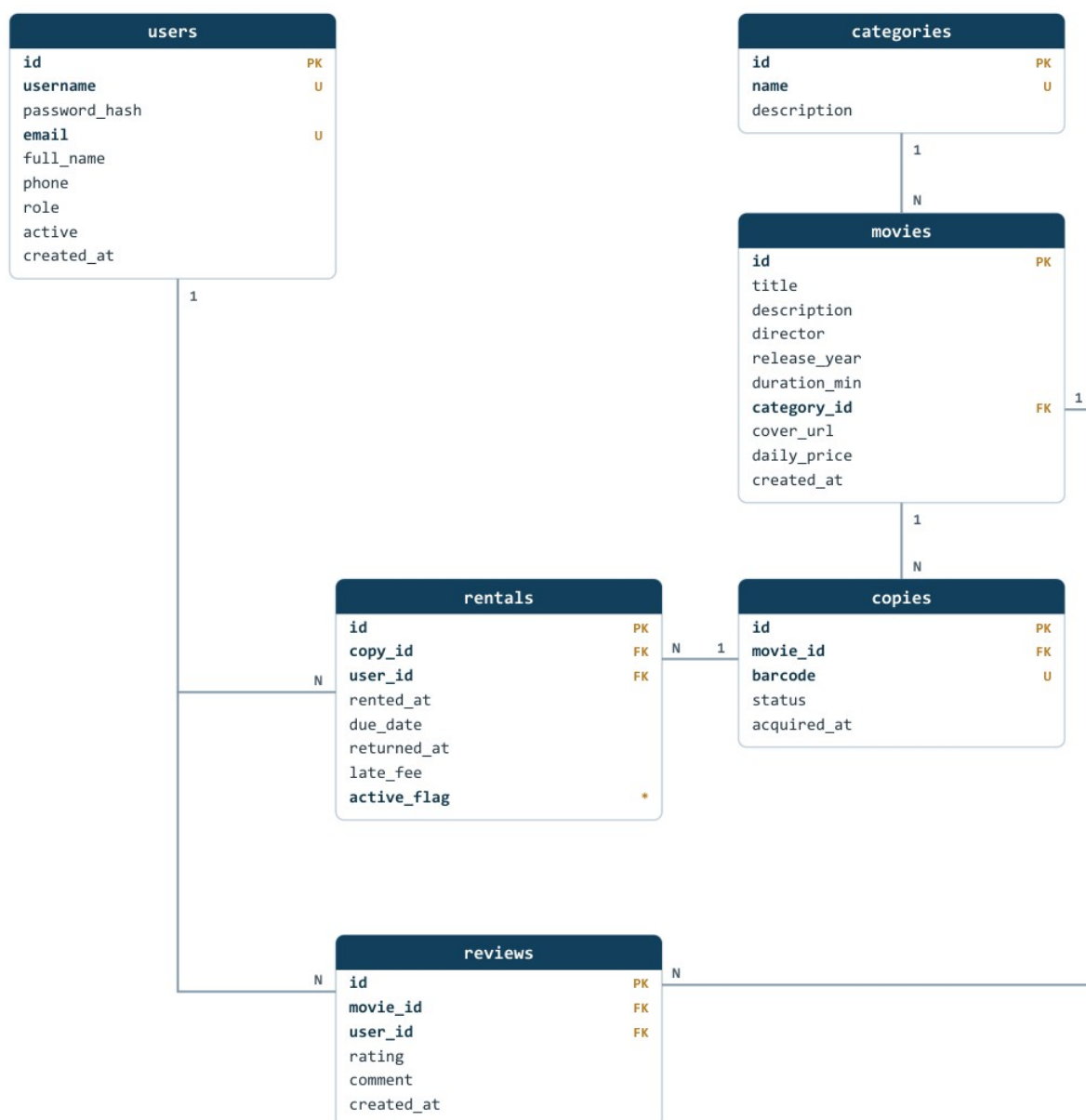
5.2 הטבלאות

טבלה	תיאור	מפתח
users	משתמשי המערכת — לקוחות ומנהלנים, כולל תמצית הסיסמה והתפקיד.	id
categories	קטגוריות הסרטים. נטענות יחד עם הסכמה.	id
movies	כותרי הקטלוג. מקושרים ל-categories וכוללים <code>cover_url</code> .	id
copies	העותקים הפיזיים. מקושרים ל-movies. זוהי היחידה המושאלת.	id
rentals	רשומת השאלה — עותק, לקוח, תאריכים וקנס.	id
reviews	ביקורות לקוחות. ייחודי לצמד (סרט, לקוח).	id

תמונות העטיפה נשמרות כנתיב ולא כתוכן: העמודה `cover_url` שומרת הפניה לתמונה. הקבצים עצמם נמצאים בנתיב `/src/main/webapp/images/covers` ונארזים לתוך ה-WAR. כך הקטלוג נשאר זמין גם ללא חיבור לאינטרנט ונשמרת האפשרות למטמון רגיל של קבצים סטטיים.

הערך ב-`cover_url` יכול להיות נתיב יחסי ליישום, למשל `images/covers/1.jpg`, או כתובת מלאה (`...://https`). שכבת התצוגה מבחינה בין שתי הצורות ומרכיבה את הכתובת הסופית.

5.3 תרשים ERD



יחסי הטבלאות. * — **active_flag** היא עמודה מחושבת המשמשת לאילוף **UNIQUE (copy_id, active_flag)**, המונע יותר מהשאלה פתוחה

תרשים ERD — reviews-I users, categories, movies, copies, rentals

5.4 העמודה **active_flag** — אכיפת השאלה פתוחה יחידה לעותק

active_flag היא עמודה מחושבת המשמשת לאילוף **UNIQUE (copy_id, active_flag)**, המונע קיום של יותר מהשאלה פתוחה אחת לעותק.

העמודה מקבלת את הערך 1 בהשאלה פתוחה ו-NULL בהשאלה שהוחזרה. אילוצי ייחודיות ב-MYSQL מאפשרים ערכי NULL מרובים, ולכן לעותק יכול להיות מספר בלתי מוגבל של השאלות שהוחזרו, אך לכל היותר השאלה פתוחה אחת.

התוצאה היא שהשאלה כפולה של אותו עותק בלתי אפשרית ברמת בסיס הנתונים, ולא רק ברמת הקוד. גם באג עתידי בשכבת השירות או שני שרתי יישומים במקביל לא יוכלו לעקוף את האילוף.

5.5 אינדקסים

אינדקס	טבלה / עמודות	מטרה
idx_copies_movie_status	copies(movie_id, status)	איתור עותק פנוי — השאילתה החמה במערכת.
idx_rentals_user	rentals(user_id)	היסטוריית לקוח.
idx_rentals_open	rentals(returned_at, due_date)	איתור השאלות פתוחות והשאלות באיחור.
active_copy_rentals_uq	rentals(copy_id, active_flag)	אכיפת השאלה פתוחה יחידה לעותק.
idx_movies_category	movies(category_id)	סינון לפי קטגוריה.

אין אינדקס FULLTEXT. ההסבר מופיע בפרק 15.3.

6. מבנה החבילות

חבילה	מספר מחלקות	אחריות
src.main.java.model	9	אובייקטי התחום
src.main.java.dao	9	גישה לנתונים וניהול טרנזקציות
src.main.java.service	4	לוגיקה עסקית
src.main.java.exception	5	חריגות התחום
src.main.java.web	9	Backing Beans, סינון הרשאות ואתחול
src.main.java.web.admin	2	מסכי הניהול
src.main.java.rest	4	משאבי JAX-RS וממפי שגיאות
src.main.java.rest.dto	4	אובייקטי תשובה ל-JSON
src.main.java.util	2	סיסמאות והגדרות

בסך הכול 48 מחלקות. חלוקת החבילות משקפת את השכבות ומאפשרת לזהות, לפי המבנה בלבד, היכן מרוכזות כל אחריות.

חבילת `src.main.java.model` מכילה אובייקטים פשוטים (POJO) המייצגים את התחום ואינם תלויים במסגרת העבודה.

User

שדות: `id, username, passwordHash, email, fullName, phone, role, active, createdAt`.

תיאור	מתודה
האם החשבון בעל הרשאות ניהול	<code>isAdmin()</code>

תמצית הסיסמה נדרשת בשכבת ה-DAO לצורך אימות, אך אינה מועברת לשכבת התצוגה.

Movie

שדות מתמידים: `title, description, director, releaseYear, durationMinutes, categoryId, coverUrl, dailyPrice, totalCopies, availableCopies, averageRating, reviewCount, categoryName`. שדות נגזרים המחושבים בשאלתה: `reviewCount, categoryName`.

תיאור	מתודה
האם קיים עותק פנוי כרגע	<code>isAvailable()</code>
דירוג לתצוגה, או null אם טרם דורג	<code>getFormattedRating()</code>

Copy

שדות: `movieId, barcode, status, acquiredAt`.

Rental

שדות: `copyId, userId, rentedAt, dueDate, returnedAt, lateFee` ושדות מצורפים לתצוגה.

תיאור	מתודה
האם העותק עדיין אצל הלקוח	<code>isOpen()</code>
האם ההשאלה פתוחה ותאריך ההחזרה כבר עבר	<code>isOverdue()</code>
מספר ימי האיחור, או 0	<code>getDaysOverdue()</code>
הקנס שייגבה אם ההחזרה תתבצע כעת	<code>getProjectedLateFee()</code>

`getProjectedLateFee()` מחזירה תמיד `BigDecimal` עם שתי ספרות אחרי הנקודה. הדבר מונע השוואות שגויות כאשר ה-`scale` שונה, למשל 0 לעומת 0.00.

CopyStatus, Role, Category, Review

אובייקטים וטיפוסי enum פשוטים. CopyStatus נושא תווית תצוגה (Available / Rented / Lost) לצד שם הקבוע, כדי שהמסך לא יציג את שם ה-enum עצמו.

MovieSearchCriteria

מיכל לקריטריוני החיפוש שהמשתמש בחר, המועבר כאובייקט אחד מדף ה-JSF ועד ל-DAO.

מתודה	תיאור
getOffset()	היסט לפסוקית LIMIT; מגן מפני היסט שלילי
hasKeyword()	האם הזן מונח חיפוש שאינו ריק
resetPaging()	חזרה לעמוד הראשון בעת שינוי סינון

העברת אובייקט קריטריונים במקום רשימת פרמטרים ארוכה שומרת על חתימת החיפוש יציבה ככל שנוספים מסננים ומאפשרת ל-DAO לבנות את פסוקית WHERE לפי השדות שמולאו בפועל.

8. שכבת הנתונים

חבילת src.main.java.dao.

Database

מחזיקה את מאגר החיבורים ומגדירה כיצד מריצים עבודה מולו.

מתודה	תיאור
init()	בונה את המאגר מקובץ ההגדרות
shutdown()	סוגר את המאגר בעת הורדת היישום
inTransaction(work)	מריץ יחידת עבודה בטרנזקציה אחת
inTransaction(isolation, work)	כנ"ל ברמת בידוד מפורשת
runInTransaction(work)	גרסה לעבודה שאינה מחזירה ערך
readOnly(work)	קריאה בלבד

גבולות הטרנזקציה מרוכזים במחלקה זו. כל יחידת עבודה מקבלת Connection; מובטח שיתבצעו commit או rollback ושהחיבור ייסגר בדיוק פעם אחת. ה-DAO מקבלים את החיבור כפרמטר ואינם פותחים חיבור בעצמם, ולכן שירות יכול לצרף כמה קריאות DAO לטרנזקציה אטומית אחת. במקרה של InnoDB, deadlock מבטל אחת הטרנזקציות. inTransaction מנסה מחדש עד שלוש פעמים עם השהיה הולכת וגדלה, ורק לאחר מכן נכשל.

רמת הבידוד היא READ COMMITTED ולא ברירת המחדל REPEATABLE READ. הנימוק מופיע בפרק 12.

runInTransaction נקראת בשם שונה ואינה overload של inTransaction, כדי למנוע עמימות בביטויי lambda שאינם מחזירים ערך.

RowMapper

ממשק פונקציונלי הממיר שורת ResultSet לאובייקט תחום, עם מתודות עזר list ו-single ומתודות קריאה המטפלות נכון ב-NULL. בעמודות מספריות נעשה שימוש ב-wasNull() כדי להבחין בין הערך 0 לבין NULL.

MovieDao

תיאור	מתודה
סרט בודד עם נתוני מלאי ודירוג	findById(conn, id)
חיפוש בקטלוג עם עימוד	search(conn, criteria)
מספר התוצאות הכולל לצורך עימוד	countMatching(conn, criteria)
תחזוקת הקטלוג	insert / update / delete

פסוקית WHERE נבנית מהקריטריונים שמולאו בפועל. רק מחרוזות SQL קבועות משורשרות; כל ערך שמקורו במשתמש מועבר כפרמטר קשור (?) ועמודת המיון נבחרת מ-enum.

נתוני המלאי copies_total ו-copies_available מחושבים בתת-שאלות ואינם נשמרים כעמודה, כדי למנוע סטייה בין מונה שמור לבין המציאות.

CopyDao

תיאור	מתודה
נועל עותק פנוי אחד עבור הטרנזקציה הקוראת	lockAvailableCopy(conn, movieId)
שינוי סטטוס מותנה במצב הקודם	updateStatus(conn, id, expected, new)
תחזוקת מלאי	findByMovie / countAvailable / insert

המתודה lockAvailableCopy מבצעת את השאלתה המרכזית למנגנון המקביליות, המתוארת בפרק 12. updateStatus משנה את השורה רק אם היא עדיין במצב הצפוי ומחזירה את מספר השורות שהשתנו.

RentalDao

תיאור	מתודה
פתיחת השאלה	insert(conn, copyId, userId, dueDate)
סגירת השאלה; מותנית ב-returned_at IS NULL	close(conn, rentalId, lateFee)
השאלות פעילות והיסטוריית ההשאלות	findOpenByUser / findByUser
מסכי הניהול	findAllOpen / findOverdue
בדיקת המכסה	countOpenByUser
האם הלקוח כבר מחזיק בכותר	hasOpenRentalOfMovie
האם רשאי לכתוב ביקורת	hasEverRentedMovie

המתודה close מותנית ב-returned_at IS NULL ולכן היא אידמפוטנטית: בקשה כפולה או שני מנהלים המטפלים באותה החזרה יעדכנו אפס שורות בפעם השנייה, במקום לדרוס את חותמת ההחזרה המקורית.

UserDao

תיאור	מתודה
שליפה	findAll / findById / findByUsername
בדיקה מקדימה להודעה ידידותית	usernameExists / emailExists
נעילת שורת הלקוח — ראו פרק 12	lockUser(conn, userId)
תחזוקה	insert / update

ReviewDao

המתודה save משתמשת ב-INSERT ... ON DUPLICATE KEY UPDATE, כך שהוספה או עדכון מתבצעים בהצהרה אחת מול אילוף הייחודיות, ללא חלון מרוץ של "קרא ואז הוסף".

DataAccessException

עוטפת SQLException כדי שהשכבות שמעל ה-DAO לא יצטרכו לייבא טיפוס JDBC.

9. שכבת הלוגיקה העסקית

חבילת src.main.java.service מסומנות ב-@ApplicationScoped. כל המחלקות מסומנות ב-@ApplicationScoped.

RentalService — לב המערכת

תיאור	מתודה
השאלת עותק — מפורט בפרק 12	rent(userId, movieId)
החזרה; מחזירה את הקנס	returnRental(rentalId, requestedBy, isAdmin)
סימון עותק כאבוד	markCopyLost(rentalId)
שאלות לקוח	openRentalsFor / historyFor
שאלות ניהול	allOpenRentals / overdueRentals
בדיקת תצוגה — האם להציג כפתור השאלה פעיל	canRent(userId, movieId)

המתודה canRent היא בדיקת תצוגה בלבד ואינה הבדיקה הקובעת: היא קוראת ללא נעילה ועלולה להתיישן. המתודה rent בודקת מחדש את כל התנאים תחת נעילה.

המתודה returnRental מקבלת גם את זהות מבקש הפעולה: לקוח רשאי להחזיר רק השאלה השייכת לו, ומנהל רשאי להחזיר כל השאלה במערכת. ניסיון להחזיר השאלה של לקוח אחר מדווח כ"לא נמצא", כדי לא לחשוף את קיומו של מזהה ההשאלה.

תיאור	מתודה
עמוד תוצאות + מספר תוצאות כולל	search(criteria)
שליפה	findMovie / categories / copiesOf
יצירת סרט + עותקים בטרנזקציה אחת	createMovie(movie, initialCopies)
תחזוקה	updateMovie / addCopies / deleteMovie

SearchResult מכילה עמוד תוצאות יחד עם מטא-נתוני העימוד. היא מחלקה עם getters ולא record, משום שמנגנון EL מחפש getX() בעוד record חושף x().

המתודה createMovie יוצרת את הסרט ואת העותקים בטרנזקציה אחת. המתודה deleteMovie מסרבת למחוק סרט כאשר קיים עותק מושאל או היסטוריית השאלות שיש לשמר.

UserService

תיאור	מתודה
יצירת חשבון לקוח	register(...)
אימות פרטים	authenticate(username, password)
ניהול	findByld / findAll / setActive

בדיקות הייחודיות המוקדמות נועדו להודעה ידידותית בלבד. גם הפרת אילוף ייחודיות בבסיס הנתונים נתפסת ומדווחת באותה צורה, כדי להתמודד עם הרשמות מקבילות.

המתודה authenticate מחשבת תמצית סיסמה גם כאשר שם המשתמש אינו קיים, כדי לצמצם דליפת מידע באמצעות הפרשי זמן.

ReviewService

המתודה submit מוודאת שהלקוח אכן שאל את הסרט בעבר. ללא כלל זה, הדירוגים המוצגים בקטלוג לא היו משקפים את ניסיונם של לקוחות ששאלו את הסרט.

חבילת החריגות

חריגה	משמעות
BusinessException	מחלקה מופשטת — בסיס לחריגות התחום
ValidationException	HTTP 400 — קלט לא תקין
AuthenticationException	HTTP 401 — אימות נכשל
NotFoundException	HTTP 404 — משאב לא נמצא
ConflictException	HTTP 409 — התנגשות מצב
DataAccessException	תקלה בגישה לנתונים, לא אירוע עסקי צפוי

הודעות חריגות עסקיות כתובות באנגלית ומיועדות להצגה ישירה למשתמש. BusinessException מייצגת תוצאה צפויה כגון "המלאי אזל"; DataAccessException מייצגת תקלה במערכת.

חבילות src.main.java.web.admin-ו src.main.java.web.

הדפים

דף	Bean	תיאור
catalog.xhtml	CatalogBean	קטלוג, חיפוש, סינון ועימוד
movie.xhtml	MovieDetailBean	פרטי סרט, השאלה וביקורות
login.xhtml	LoginBean	כניסה
register.xhtml	RegisterBean	הרשמה
my-rentals.xhtml	MyRentalsBean	השאלות והיסטוריה
admin/movies.xhtml	MovieAdminBean	ניהול קטלוג ומלאי
admin/returns.xhtml	ReturnsBean	השאלות פתוחות והחזרות
error.xhtml	—	דף שגיאה אחיד
WEB-INF/templates/layout.xhtml	—	תבנית משותפת

תחומי החיים של ה-Beans

תחום	מחלקות	נימוק
@SessionScoped	SessionBean	זהות המשתמש נשמרת לאורך הביקור
@ViewScoped	CatalogBean, MovieDetailBean, MyRentalsBean, MovieAdminBean, ReturnsBean	המצב נשמר בין postbacks ונעלם ביציאה מהמסך
@RequestScoped	LoginBean, RegisterBean	טופס חד-פעמי

@ViewScoped נבחר לקטלוג משום ש-@RequestScoped היה מאפס את המסננים בכל postback, בעוד @SessionScoped היה משאיר אותם גם לאחר מעבר למסכים אחרים.

SessionBean

מחזיקה את זהות המשתמש. שם ה-Bean הוא userSession כדי להימנע מהתנגשות עם producer מובנה בשם session.

login() מחליפה את מזהה ה-Session באמצעות changeSessionId() (כהגנה מפני Session Fixation). המימוש הראשון שביטל את ה-Session באמצעות invalidate() גרם גם להשמדת ה-Bean עצמו; changeSessionId() מחליף את המזהה תוך שמירת תוכן ה-Session.

AuthFilter

Servlet Filter המגן על my-rentals.xhtml ועל */admin/. האכיפה נמצאת ב-Filter ולא בכל דף, כדי שכל דף חדש תחת נתיב מוגן יהיה מוגן מעצם כתובתו. משתמש שאינו מחובר מופנה למסך הכניסה עם היעד המקורי בפרמטר returnUrl.

StartupListener

מאתחל את מאגר החיבורים בעת עליית היישום וסוגר אותו בעת עצירתו, כך שמשאבים משתחררים גם בפריסה מחדש.

11. שכבת ה-REST

חבילות src.main.java.rest.dto-I src.main.java.rest.

מחלקה	תפקיד
RestApplication	קובעת את שורש הממשק ל-/api
MovieResource	חיפוש, פרטי סרט וביקורות
CategoryResource	רשימת קטגוריות
ErrorMappers	מיפוי חריגות לקודי HTTP

אובייקטי ה-DTO

PageDto, MovieDto, CategoryDto, ReviewDto הם record. הממשק מחזיר DTO ולא אובייקטי תחום כדי שהפורמט הציבורי יהיה החלטה מפורשת ולא תוצר לוואי של סכמת הנתונים. ReviewDto נושא את שם המבקר אך לא את מזהו.

הגבלת הממשק לקריאה בלבד

פעולת השאלה משנה מלאי ומחייבת זיהוי משתמש. חשיפתה ב-REST הייתה מחייבת מנגנון הזדהות נוסף לצד ה-Session של דפי ה-JSF; לכן הממשק הציבורי מוגבל לפעולות קריאה.

12. טיפול במקביליות

זהו הפרק המרכזי במסמך. הדרישה "המערכת לא תשאל סרט שעדיין לא הוחזר" מחייבת הגנה מפני מצבי מרוץ אמיתיים בין בקשות מקבילות.

12.1 שני מרוצים שונים

המרוץ	התרחיש	ההגנה
לקוחות שונים, אותו כותר	20 לקוחות לוחצים Rent על הסרט האחרון	נעילת שורת העותק + אילוץ ייחודיות
לקוח אחד, בקשות מקבילות	לחיצה כפולה או שתי לשוניות פתוחות	נעילת שורת הלקוח

12.2 מהלך פעולת ההשאלה

RentalService.rent() מתבצעת בתוך טרנזקציה אחת:

#	הפעולה	הקריאה	מה היא מגנה מפניו
1	נעילת שורת הלקוח	userDao.lockUser()	בקשות מקבילות של אותו לקוח
2	שליפת הסרט	movieDao.findById()	—

3	בדיקת מכסת ההשאלות	rentalDao.countOpenByUser()	חריגה מהמכסה
4	בדיקה שהכותר אינו נמצא כבר אצלו	rentalDao.hasOpenRentalOfMovie()	השאלה כפולה של אותו כותר
5	נעילת עותק פנוי	copyDao.lockAvailableCopy()	לקוחות שונים על אותו עותק
6	שינוי סטטוס ל-RENTED	copyDao.updateStatus()	עדכון מותנה במצב קודם
7	יצירת רשומת ההשאלה	rentalDao.insert()	אילוץ הייחודיות במסד הנתונים

כל השלבים מבוצעים יחד או כלל לא.

12.3 נעילת העותק

```
SELECT id FROM copies
WHERE movie_id = ? AND status = 'AVAILABLE'
ORDER BY id LIMIT 1
FOR UPDATE SKIP LOCKED
```

- **FOR UPDATE** — נעילה בלעדית על השורה, המוחזקת עד סיום הטרנזקציה. טרנזקציה אחרת אינה יכולה לבחור את אותה שורה כעותק פנוי.
- **SKIP LOCKED** — דילוג על שורות שכבר נעולות במקום להמתין להן. כאשר קיימים כמה עותקים פנויים, בקשות מקבילות יכולות לבחור עותקים שונים.

12.4 נעילת הלקוח ורמת הבידוד

החוקים המבוססים על ספירה לכל לקוח — מכסת ההשאלות והאיסור להשאל כותר שכבר נמצא אצלו — אינם יכולים להתבסס על "קרא ואז פעל" כאשר שתי בקשות מגיעות יחד. הפתרון הוא נעילת שורת הלקוח כהצהרה הראשונה בטרנזקציה.

- סדר הנעילות — תמיד לקוח ואז עותק. סדר אחיד בכל הטרנזקציות מצמצם סיכון לקיפאון הדדי.
- רמת הבידוד — טרנזקציות המערכת רצות ב-**READ COMMITTED**, כך שכל הצהרה קוראת מצב מעודכן לאחר הנעילה. ברירת המחדל **REPEATABLE READ** עלולה להחזיק snapshot ישן עבור קריאות שאינן נועלות.

בדיקת `doubleSubmitByOneCustomerRentsOnce` גילתה שמימוש מוקדם אפשר ללקוח יחיד לקבל שני עותקים של אותו סרט. הוספת נעילת הלקוח לבדה לא הספיקה כאשר קריאות נעשו לפני הנעילה. שינוי סדר הפעולות יחד עם **READ COMMITTED** פתר את הבעיה.

12.5 שלוש שכבות הגנה

השכבה	מה היא מונעת
נעילת שורת הלקוח	בקשות מקבילות של אותו לקוח
נעילת שורת העותק	לקוחות שונים התופסים את אותו עותק
active_copy_rentals_uq אילוץ הייחודיות	כל השאר — כולל באג עתידי בקוד

ההגנה השלישית היא הבטחה ברמת בסיס הנתונים ואינה תלויה בנכונות הקוד. גם `updateStatus` מותנית ב-`status = AVAILABLE`, כך שכשל יתגלה במפורש ולא יעבור בשקט.

האיום	ההגנה
הזרקת SQL	PreparedStatement בכל שאילתה; רק מחרוזות קבועות משורשרות; עמודת מיון נבחרת מ-enum
גניבת סיסמאות ממאגר גנוב	סבבים, מלח אקראי PBKDF2-HMAC-SHA256, 210,000 לכל משתמש
התקפת תזמון על שם משתמש	חישוב תמצית גם עבור משתמש שאינו קיים
חשיפת שמות משתמש קיימים	הודעת כניסה אחידה
Session Fixation	בעת ההתחברות changeSessionId()
גניבת עוגיית Session באמצעות סקריפט	Session-על עוגיית ה HttpOnly
XSS	JSF מקודד פלט אוטומטית
גישה לא מורשית	AuthFilter לפי כתובת, ולא הסתרת קישורים
חשיפת מבנה פנימי בשגיאה	דף שגיאה כללי; הפרטים ליומן בלבד

PBKDF2 נבחר משום שאלגוריתם hash מהיר אינו מתאים לאחסון סיסמאות. 210,000 סבבים מעלים במכון את עלות כל ניחוש, והמלח האקראי מבטיח ששני משתמשים שבחרו אותה סיסמה יקבלו תמציות שונות.

ההשוואה נעשית באמצעות MessageDigest.isEqual(), הבודקת את כל הבתים ומפחיתה דליפת מידע דרך הפרשי זמן.

14. החלטות תכנון והנימוק להן

14.1 סרט מול עותק

ניתן היה לשמור מונה count_available על טבלת הסרטים, אך מונה כזה מחייב עדכון בכל השאלה והחזרה ואינו מתעד איזה דיסק נמצא אצל כל לקוח. מודל העותקים נותן מעקב מדויק וזמינות נגזרת שאינה יכולה לסטות.

14.2 JPA ולא JDB

הבחירה מכוונת. המנגנון המרכזי הוא SELECT ... FOR UPDATE SKIP LOCKED — נעילה מפורשת שהיא לב פתרון המקביליות. ניסוח הנעילה ישירות ב-SQL בהיר ומדויק יותר עבור הפרויקט, וגם הופך את גבולות הטרנזקציה למפורשים.

14.3 העדר אינדקס FULLTEXT

ההחלטה התקבלה לאחר ניסוי. FULLTEXT ב-MYSQL מתאים לחיפוש מילים שלמות או תחיליות ואינו מתאים לחיפוש קטע מאמצע מילה. לכן חיפוש חלקי כמו budap לא היה בהכרח מגיע ל-The Grand Budapest Hotel באמצעות חיפוש תחילית, בעוד התאמת תת-מחרוזת כן:

אינו מחזיר קטע מאמצע מילה --

```
SELECT ... WHERE MATCH(title, ...) AGAINST ('+budap*' IN BOOLEAN MODE);
```

מחזיר את השורה --

```
SELECT ... WHERE title LIKE '%budap%';
```

לכן החיפוש מבוסס על התאמת תת-מחרוזת. בקטלוג בסדר הגודל של הפרויקט עלות הסריקה אינה משמעותית. אילו הקטלוג היה גדל לעשרות אלפי כותרים, הפתרון המתאים היה מנוע חיפוש ייעודי.

14.4 חוקים עסקיים כהגדרות

תקופת ההשאלה, המכסה וקנס האיחור נמצאים ב-sartia.properties, וניתן לשנות את ערכיהם באמצעות פרמטר הרצה. אלה ערכי מדיניות, ושינוי שלהם אינו אמור לחייב בנייה מחדש.

14.5 עמידות בפני קלט לא תקין

הקלט נבדק במכוון בשלוש נקודות:

- בדפדפן — סימון שדות חובה. לנוחות בלבד; אין בכך הגנה.
- בשכבת השירות — בדיקות תוכן. זו האכיפה העסקית.
- בבסיס הנתונים — אילוצים, מפתחות זרים ו-CHECK. שכבת ההגנה האחרונה.